```
print("Experiment 8")
print("Parth Mahesh Madrewar")
print("Reg. No. : 241080036")
```

```
Experiment 8
Parth Mahesh Madrewar
Reg. No. : 241080036
```

```
import time
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, confusion_matrix
from sklearn.svm import SVC
from sklearn.tree import DecisionTreeClassifier
from sklearn.neural_network import MLPClassifier
```

```
data = pd.read_csv("/content/heart_cleveland_upload.csv")
print(data.head())
print("Shape:", data.shape)
print(data.isnull().sum())
```

```
   age  sex  cp  trestbps  chol  fbs  restecg  thalach  exang  oldpeak  slope  \
0   69    1   0       160   234    1        2      131      0      0.1      1
1   69    0   0       140   239    0        0      151      0      1.8      0
2   66    0   0       150   226    0        0      114      0      2.6      2
3   65    1   0       138   282    1        2      174      0      1.4      1
4   64    1   0       110   211    0        2      144      1      1.8      1

   ca  thal  condition
0   1     0          0
1   2     0          0
2   0     0          0
3   1     0          1
4   0     0          0
Shape: (297, 14)
age          0
sex          0
cp           0
trestbps     0
chol         0
fbs          0
restecg      0
thalach      0
exang        0
oldpeak      0
slope        0
ca           0
thal         0
condition    0
dtype: int64
```

```
X = data.drop("condition", axis=1)
y = data["condition"]

X = X.astype(float)
y = y.astype(float)

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# SVM needs labels as -1 and +1 instead of 0 and 1
y_train_svm = np.where(y_train == 1, 1, -1)

print("Training samples:", len(X_train))
print("Testing samples:", len(X_test))
```

```
Training samples: 237
Testing samples: 60
```

```
class LinearSVMScratch:
    def __init__(self, learning_rate=0.1, lambda_param=0.01, epochs=1000):
        self.learning_rate = learning_rate
        self.lambda_param = lambda_param       # regularization, C = 1 / lambda
        self.epochs = epochs
        self.weights = None
        self.bias = 0
```

```
        self.bias = 0
        self.loss = []

    def fit(self, X, y):
        rows, columns = X.shape
        self.weights = np.zeros(columns)
        self.bias = 0

        for i in range(self.epochs):
            margin = y * (np.dot(X, self.weights) + self.bias)
            active = margin < 1                # points inside or on the wrong side of the margin

            # hinge loss + L2 penalty
            hinge = np.mean(np.maximum(0, 1 - margin))
            self.loss.append(self.lambda_param * np.dot(self.weights, self.weights) + hinge)      #-> also known a

            weight_gradient = 2 * self.lambda_param * self.weights - np.dot(X[active].T, y[active]) / rows
            bias_gradient = -np.sum(y[active]) / rows

            self.weights = self.weights - self.learning_rate * weight_gradient
            self.bias = self.bias - self.learning_rate * bias_gradient

    def decision_function(self, X):
        return np.dot(X, self.weights) + self.bias

    def predict(self, X):
        return (self.decision_function(X) >= 0).astype(int)
```

```
model = LinearSVMScratch(learning_rate=0.1, lambda_param=0.01, epochs=1000)
model.fit(X_train, y_train_svm)
y_pred = model.predict(X_test)

print("\nBias:")
print(model.bias)
print("\nWeights:")
print(model.weights)

# Performance
acc = accuracy_score(y_test, y_pred)
cm = confusion_matrix(y_test, y_pred)
print("\nScratch Linear SVM")
print("Accuracy :", acc)
print("Confusion Matrix:\n", cm)

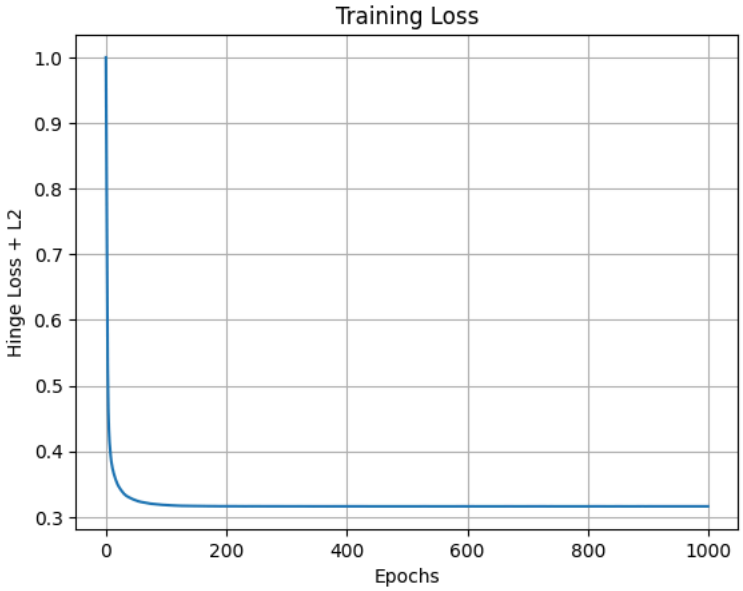
# Support vectors -> points with margin <= 1
margins = y_train_svm * model.decision_function(X_train)
sv_index = np.where(margins <= 1)[0]
print("Number of Support Vectors:", len(sv_index))
print("Support Vector indices:\n", sv_index)

# Plot training loss
plt.plot(model.loss)
plt.title("Training Loss")
plt.xlabel("Epochs")
plt.ylabel("Hinge Loss + L2")
plt.grid()
plt.show()
```

```
Bias:
-0.10084388185653992

Weights:
[-0.05759377  0.5031879   0.32198329  0.23553988  0.1347891  -0.22990238
  0.13986192 -0.50247142  0.15010589  0.2248608   0.1759515   0.59011448
  0.58808661]

Scratch Linear SVM
Accuracy : 0.75
Confusion Matrix:
 [[23  9]
 [ 6 22]]
Number of Support Vectors: 76
Support Vector indices:
 [  1   8  10  13  19  21  26  34  35  37  39  41  44  45  46  47  51  53
  54  55  60  61  62  64  68  70  71  72  77  79  88  97  98  99 104 112
 114 117 121 124 131 132 138 140 141 142 150 156 159 164 166 167 171 175
 179 182 185 186 188 194 195 196 198 201 202 213 218 220 222 226 227 229
 230 231 232 234]
```




```
# Linear SVM using Scikit-Learn
model2 = SVC(kernel="linear", C=1.0)
model2.fit(X_train, y_train)
y_pred2 = model2.predict(X_test)

# Performance
acc2 = accuracy_score(y_test, y_pred2)
cm2 = confusion_matrix(y_test, y_pred2)
print("Scikit-Learn Linear SVM")
print("Accuracy :", acc2)
print("Confusion Matrix:\n", cm2)
print("Number of Support Vectors:", len(model2.support_))
```

```
Scikit-Learn Linear SVM
Accuracy : 0.75
Confusion Matrix:
 [[23  9]
 [ 6 22]]
Number of Support Vectors: 79
```

```
class KernelSVMScratch:
    def __init__(self, kernel="rbf", C=1.0, gamma=0.05, epochs=2000):
        self.kernel = kernel
        self.C = C
        self.gamma = gamma
        self.epochs = epochs
        self.alpha = None
        self.X_train = None
        self.y_train = None
        self.loss = []

    def kernel_function(self, X1, X2):
        if self.kernel == "linear":
            k = np.dot(X1, X2.T)
        else:
            # ||a - b||^2 = a.a + b.b - 2 a.b
```

```
            sq_dist = np.sum(X1 ** 2, axis=1).reshape(-1, 1) + np.sum(X2 ** 2, axis=1) - 2 * np.dot(X1, X2.T)
            k = np.exp(-self.gamma * sq_dist)
        return k + 1        # +1 absorbs the bias term

    def fit(self, X, y):
        self.X_train = X
        self.y_train = y

        K = self.kernel_function(X, X)
        Q = np.outer(y, y) * K

        # step size 1/L (L = largest eigenvalue of Q) keeps the updates stable
        learning_rate = 1 / np.max(np.linalg.eigvalsh(Q))
        alpha = np.zeros(len(y))

        for i in range(self.epochs):
            gradient = 1 - np.dot(Q, alpha)
            alpha = np.clip(alpha + learning_rate * gradient, 0, self.C)     # keep 0 <= alpha <= C

            dual = np.sum(alpha) - 0.5 * np.dot(alpha, np.dot(Q, alpha))
            self.loss.append(-dual)         #-> negative dual objective, we minimize this

        self.alpha = alpha

    def decision_function(self, X):
        return np.dot(self.kernel_function(X, self.X_train), self.alpha * self.y_train)

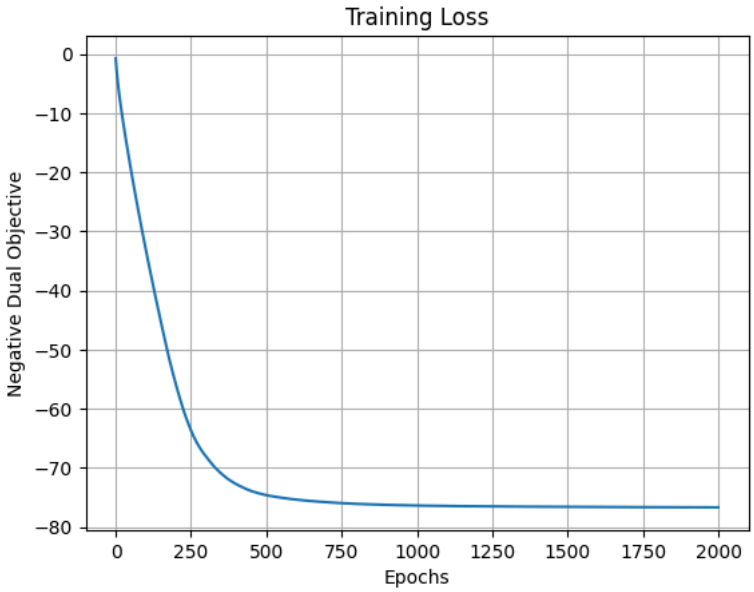
    def predict(self, X):
        return (self.decision_function(X) >= 0).astype(int)
```

```
model3 = KernelSVMScratch(kernel="rbf", C=1.0, gamma=0.05, epochs=2000)
model3.fit(X_train, y_train_svm)
y_pred3 = model3.predict(X_test)

# Performance
acc3 = accuracy_score(y_test, y_pred3)
cm3 = confusion_matrix(y_test, y_pred3)
sv_index3 = np.where(model3.alpha > 1e-5)[0]
print("Scratch Kernel SVM (RBF)")
print("Accuracy :", acc3)
print("Confusion Matrix:\n", cm3)
print("Number of Support Vectors:", len(sv_index3))
print("Support Vector indices:\n", sv_index3)

# Plot training loss
plt.plot(model3.loss)
plt.title("Training Loss")
plt.xlabel("Epochs")
plt.ylabel("Negative Dual Objective")
plt.grid()
plt.show()
```

```
Scratch Kernel SVM (RBF)
Accuracy : 0.7333333333333333
Confusion Matrix:
 [[22 10]
 [ 6 22]]
Number of Support Vectors: 124
Support Vector indices:
 [  1   8  10  12  13  14  19  21  22  24  26  32  34  35  37  39  41  43
  44  45  46  47  48  49  51  53  54  55  58  59  60  61  62  64  65  67
  68  70  71  72  73  76  77  79  80  86  88  89  90  92  95  96  97  98
  99 103 104 106 112 114 117 121 124 128 131 132 135 136 138 140 141 142
 145 147 150 152 153 156 157 159 160 161 164 165 166 167 169 171 172 175
 176 179 182 184 185 186 188 189 190 191 193 194 195 196 197 198 201 202
 203 205 209 210 213 218 220 222 226 227 229 230 231 232 234 235]
```




```
# Kernel SVM using Scikit-Learn
model4 = SVC(kernel="rbf", C=1.0, gamma=0.05)
model4.fit(X_train, y_train)
y_pred4 = model4.predict(X_test)

# Performance
acc4 = accuracy_score(y_test, y_pred4)
cm4 = confusion_matrix(y_test, y_pred4)
print("Scikit-Learn Kernel SVM (RBF)")
print("Accuracy :", acc4)
print("Confusion Matrix:\n", cm4)
print("Number of Support Vectors:", len(model4.support_))
print("Support Vector indices:\n", model4.support_)
```

```
Scikit-Learn Kernel SVM (RBF)
Accuracy : 0.75
Confusion Matrix:
 [[22 10]
 [ 5 23]]
Number of Support Vectors: 118
Support Vector indices:
 [ 13  19  21  24  32  34  37  41  43  45  47  48  54  55  60  61  62  65
  71  73  76  77  80  86  88 106 117 121 136 142 145 147 150 152 153 156
 157 159 160 164 165 166 171 172 175 176 179 186 189 190 201 202 205 213
 220 226 229 231 232   1   8  10  12  14  22  26  35  39  44  46  49  51
  53  58  59  64  67  68  70  72  79  89  90  92  95  97  98  99 103 104
 112 114 124 131 132 138 140 141 161 167 169 182 184 185 188 194 195 196
 197 198 203 210 218 222 227 230 234 235]
```

```
# Decision Tree and Neural Network using Scikit-Learn
start = time.time()
tree_model = DecisionTreeClassifier(criterion="entropy", max_depth=5, random_state=42)
tree_model.fit(X_train, y_train)
tree_time = time.time() - start
y_pred_tree = tree_model.predict(X_test)
tree_accuracy = accuracy_score(y_test, y_pred_tree)

start = time.time()
nn_model = MLPClassifier(hidden_layer_sizes=(64, 32), max_iter=500, random_state=42)
nn_model.fit(X_train, y_train)
nn_time = time.time() - start
y_pred_nn = nn_model.predict(X_test)
nn_accuracy = accuracy_score(y_test, y_pred_nn)
```

```
print("Scikit-Learn Decision Tree")
print("Accuracy :", tree_accuracy)
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred_tree))

print("\nScikit-Learn Neural Network (MLP)")
print("Accuracy :", nn_accuracy)
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred_nn))
```

```
Scikit-Learn Decision Tree
Accuracy : 0.7
Confusion Matrix:
```